Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by: Sharon Mathew

Verified by:

Dr. Rupali Sunil Wagh
Dr. Helen K Joy
Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto "Excellence and Service," and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
PO4: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
Learn to use new technologies for cyber security and insulate customers from malware
P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING
Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75
L + P + T: 3 + 4 + 0
Max Marks: 100
Credits: 4
Course Type: Core Course
Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS Teaching Hours: 15 Hours

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION Teaching Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow for designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a Multi-layer perceptron
13. Deploying Trained ML-models

Text and Reference Books

[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.

[2] Andreas C. Müller and Sarah Guido (2017), "Introduction to Machine Learning with Python A Guide For Data Scientists" O'Reilly book

[3] Ethem Alpaydin (2005), "Introduction to Machine Learning", Prentice Hall of India

[4] Max Kuhn and Kjell Johnson (2018), "Applied Predictive Modelling", Springer

Essential Reading

[1] Stuart Russell and Peter Norvig(2020), "Artificial Intelligence: A Modern Approach" (4th Edition), Pearson

[2] Aurelien Geron(2019), "Hands on Machine Learning with Scikit-learn, Keras and Tensorflow", 2nd Edition, O'Reilly Books

[3] Kevin P. Murphy(2012), "Machine Learning: A Probabilistic Perspective", MIT Press

[4] Hastie, Tibshirani, Friedman(2008), "The Elements of Statistical Learning" (2nd ed), Springer

[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), "Deep Learning (Adaptive Computation And Machine Learning Series)", MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera: https://www.deeplearning.ai/courses/machine-learning-specialization/

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MCA-B

Sl. no	Title of lab Experiment	Page number	RBT	CO
1	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration Combined Lab Sheet	8	L3	CO1, 3
2	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration Combined Lab Sheet	8	L3	CO2, 3
3	Student Survey: Simple Linear Regression using Scikit Learn, OLS, and Parameter Saving		L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.

- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I:

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

TASK 1:

A junior analyst hands you two CSV files and says — "I have no idea what's in these. We need to use them to build a machine learning model next week." Before any analysis can begin, you need to understand what you are working with.

Investigate both files thoroughly. Produce a structured summary that gives a complete first-impression picture of the data — its size, its contents, and where it might be problematic. Decide what information a data scientist would need before trusting this data, and make sure your summary covers all of it.

A structured data profile for each file in a markdown cell
At least one written observation about what concerns you after the inspection
Think: What does a data scientist need to know about a dataset before using it? What functions help you find that?

CODE:

```
import pandas as pd
import numpy as np



city_df = pd.read_csv("city_day.csv")
crop_df = pd.read_csv("crop_production.csv")



def data_profile(df, dataset_name):
    print("\n" + "="*80)
    print(f"DATA PROFILE : {dataset_name}")
    print("="*80)

    print("\n1. Dataset Shape")
    print(f"Rows    : {df.shape[0]}")
    print(f"Columns : {df.shape[1]}")
```

```
    print("\n2. Data Types")
    print(df.dtypes)

    print("\n3. First 5 Records")
    print(df.head())

    print("\n4. Missing Values")
    missing = pd.DataFrame({
        'Missing_Count': df.isnull().sum(),
        'Missing_%': round((df.isnull().sum()/len(df))*100,2)
    }).sort_values(by='Missing_%', ascending=False)

    print(missing[missing['Missing_Count'] > 0])

    print("\n6. Duplicate Rows")
    print(df.duplicated().sum())

    print("\n7. Numerical Summary")
    print(df.describe())

    print("\n8. Categorical Summary")
    print(df.describe(include='object'))

    print("\n9. Unique Values")
    print(df.nunique())

# Run Profiles
data_profile(city_df, "City Air Quality Dataset")
data_profile(crop_df, "Crop Production Dataset")
```

RESULT

```
================================================================================
DATA PROFILE : City Air Quality Dataset
================================================================================

1. Dataset Shape
Rows    : 29531
Columns : 16

2. Data Types
City           object
Date           object
PM2.5         float64
PM10          float64
NO            float64
NO2           float64
NOx           float64
NH3           float64
CO            float64
SO2           float64
O3            float64
Benzene       float64
Toluene       float64
Xylene        float64
AQI           float64
AQI_Bucket     object
dtype: object
```

```
3. First 5 Records
        City        Date  PM2.5  PM10     NO    NO2    NOx  NH3     CO    SO2  \
0  Ahmedabad  2015-01-01    NaN   NaN   0.92  18.22  17.15  NaN   0.92  27.64
1  Ahmedabad  2015-01-02    NaN   NaN   0.97  15.69  16.46  NaN   0.97  24.55
2  Ahmedabad  2015-01-03    NaN   NaN  17.40  19.30  29.70  NaN  17.40  29.07
3  Ahmedabad  2015-01-04    NaN   NaN   1.70  18.48  17.97  NaN   1.70  18.59
4  Ahmedabad  2015-01-05    NaN   NaN  22.10  21.42  37.76  NaN  22.10  39.33

       O3  Benzene  Toluene  Xylene  AQI AQI_Bucket
0  133.36     0.00     0.02    0.00  NaN        NaN
1   34.06     3.68     5.50    3.77  NaN        NaN
2   30.70     6.80    16.40    2.25  NaN        NaN
3   36.08     4.43    10.14    1.00  NaN        NaN
4   39.31     7.01    18.89    2.78  NaN        NaN
```

```
4. Missing Values
            Missing_Count  Missing_%
Xylene              18109      61.32
PM10                11140      37.72
NH3                 10328      34.97
Toluene              8041      27.23
Benzene              5623      19.04
AQI                  4681      15.85
AQI_Bucket           4681      15.85
PM2.5                4598      15.57
NOx                  4185      14.17
O3                   4022      13.62
SO2                  3854      13.05
NO2                  3585      12.14
NO                   3582      12.13
CO                   2059       6.97

6. Duplicate Rows
0
```

```
8. Categorical Summary
             City        Date AQI_Bucket
count       29531       29531      24850
unique         26        2009          6
top     Ahmedabad  2020-06-26   Moderate
freq         2009          26       8829

9. Unique Values
City             26
Date           2009
PM2.5         11716
PM10          12571
NO             5776
NO2            7404
NOx            8156
NH3            5922
CO             1779
SO2            4761
O3             7699
Benzene        1873
Toluene        3608
Xylene         1561
AQI             829
AQI_Bucket        6
dtype: int64
```

```
7. Numerical Summary
           Crop_Year          Area    Production
count  246091.000000  2.460910e+05  2.423610e+05
mean     2005.643018  1.200282e+04  5.825034e+05
std         4.952164  5.052340e+04  1.706581e+07
min      1997.000000  4.000000e-02  0.000000e+00
25%      2002.000000  8.000000e+01  8.800000e+01
50%      2006.000000  5.820000e+02  7.290000e+02
75%      2010.000000  4.392000e+03  7.023000e+03
max      2015.000000  8.580100e+06  1.250800e+09

8. Categorical Summary
           State_Name District_Name       Season    Crop
count          246091        246091       246091  246091
unique             33           646            6     124
top     Uttar Pradesh       BIJAPUR  Kharif         Rice
freq            33306           945        95951   15104
```

TASK 2:

After the inspection, it is clear that several columns have missing values. A colleague suggests simply deleting all rows with any null value. Another suggests filling everything with the mean. You are not sure either approach is right.
Devise your own missing value treatment strategy. For each column with missing data, decide whether to drop it, drop affected rows, or impute — and explain why that choice is appropriate for that specific column. Apply your strategy and verify it worked.

A markdown cell that justifies each decision column by column
Before and after null counts as evidence the treatment worked.

```
city_df.drop(columns=['Xylene'], inplace=True)



num_cols = [
    'PM2.5','PM10','NO','NO2','NOx',
    'NH3','CO','SO2','O3',
    'Benzene','Toluene','AQI'
]


for col in num_cols:
    city_df[col].fillna(city_df[col].median(), inplace=True)


city_df['AQI_Bucket'].fillna(
    city_df['AQI_Bucket'].mode()[0],
    inplace=True
)



print("Missing Values After Treatment")
print(city_df.isnull().sum())


print("\nTotal Missing Values:",
```

```
    city_df.isnull().sum().sum())
```

RESULT:

```
Missing Values After Treatment
City          0
Date          0
PM2.5         0
PM10          0
NO            0
NO2           0
NOx           0
NH3           0
CO            0
SO2           0
O3            0
Benzene       0
Toluene       0
AQI           0
AQI_Bucket    0
dtype: int64
```

TASK 3:

You need to combine both files later in the lab using the State column as the common key. But when you look closely, the same state appears with different spellings, extra spaces, or old names across the two files. There are also rows that appear more than once for no clear reason.
Make both files agree on state names and remove any redundant records. Document every inconsistency you found and how you resolved it. Show that the files are now ready to be merged reliably.

A list of all inconsistencies found and the fix applied for each
Record counts before and after to prove duplicates were removed

```
print("Before Treatment")
print(crop_df.isnull().sum())


# --------------------
# Remove rows with missing Production
# --------------------
crop_df = crop_df.dropna(subset=['Production'])


# --------------------
# Verification
# --------------------
print("\nAfter Treatment")
print(crop_df.isnull().sum())


print("\nTotal Missing Values:",
      crop_df.isnull().sum().sum())
```

RESULT:

```
Before Treatment
State_Name          0
District_Name       0
Crop_Year           0
Season              0
Crop                0
Area                0
Production       3730
dtype: int64

After Treatment
State_Name       0
District_Name    0
Crop_Year        0
Season           0
Crop             0
Area             0
Production       0
dtype: int64

Total Missing Values: 0
```

```
print("Unique States:")
print(sorted(crop_df['State_Name'].unique()))
```

Result: ['Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir ', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana ', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']

```
def clean_state_name(state):


    if pd.isna(state):
        return state


    state = str(state).strip().upper()
```

```
    replacements = {
        'TAMILNADU': 'TAMIL NADU',
        'TAMIL NADU ': 'TAMIL NADU',
        'PONDICHERRY': 'PUDUCHERRY',
        'ORISSA': 'ODISHA',
        'UTTARANCHAL': 'UTTARAKHAND',
        'ANDAMAN AND NICOBAR ISLANDS':
            'ANDAMAN & NICOBAR ISLANDS'
    }

    return replacements.get(state, state)

crop_df['State_Name'] = crop_df['State_Name'].apply(clean_state_name)
```

```
states_before = sorted(
    pd.read_csv("crop_production.csv")
    ['State_Name']
    .dropna()
    .unique()
)

print(states_before)

states_after = sorted(
    crop_df['State_Name']
    .dropna()
    .unique()
)
```

```
print(states_after)
```

Result: ['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir ', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana ', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']

['ANDAMAN & NICOBAR ISLANDS', 'ANDHRA PRADESH', 'ARUNACHAL PRADESH', 'ASSAM', 'BIHAR', 'CHANDIGARH', 'CHHATTISGARH', 'DADRA AND NAGAR HAVELI', 'GOA', 'GUJARAT', 'HARYANA', 'HIMACHAL PRADESH', 'JAMMU AND KASHMIR', 'JHARKHAND', 'KARNATAKA', 'KERALA', 'MADHYA PRADESH', 'MAHARASHTRA', 'MANIPUR', 'MEGHALAYA', 'MIZORAM', 'NAGALAND', 'ODISHA', 'PUDUCHERRY', 'PUNJAB', 'RAJASTHAN', 'SIKKIM', 'TAMIL NADU', 'TELANGANA', 'TRIPURA', 'UTTAR PRADESH', 'UTTARAKHAND', 'WEST BENGAL']

```
before_crop = len(crop_df)


crop_df = crop_df.drop_duplicates()


after_crop = len(crop_df)


print("Crop Dataset")
print("Before:", before_crop)
print("After :", after_crop)
print("Duplicates Removed:",
      before_crop - after_crop)


city_df = pd.read_csv("city_day.csv")


before_city = len(city_df)
```

```
city_df = city_df.drop_duplicates()

after_city = len(city_df)

print("City Dataset")
print("Before:", before_city)
print("After :", after_city)
print("Duplicates Removed:",
      before_city - after_city)

print("City duplicates:",
      city_df.duplicated().sum())

print("Crop duplicates:",
      crop_df.duplicated().sum())

city_df.to_csv(
    "city_day_cleaned.csv",
    index=False
)

crop_df.to_csv(
    "crop_production_cleaned.csv",
    index=False
)
```

Result:

```
Crop Dataset
Before: 246091
After : 246091
Duplicates Removed: 0
City Dataset
Before: 29531
After : 29531
Duplicates Removed: 0
City duplicates: 0
Crop duplicates: 0
```

TASK 4:

The pollution control board wants to know — are most Indian cities moderately polluted, or is the problem concentrated in just a few places? They also want to know whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.

Investigate the shape of the AQI distribution. Decide which visualisation(s) best answer both questions the board is asking, justify your choice, produce the plot(s), and record two specific observations that directly address their concerns.

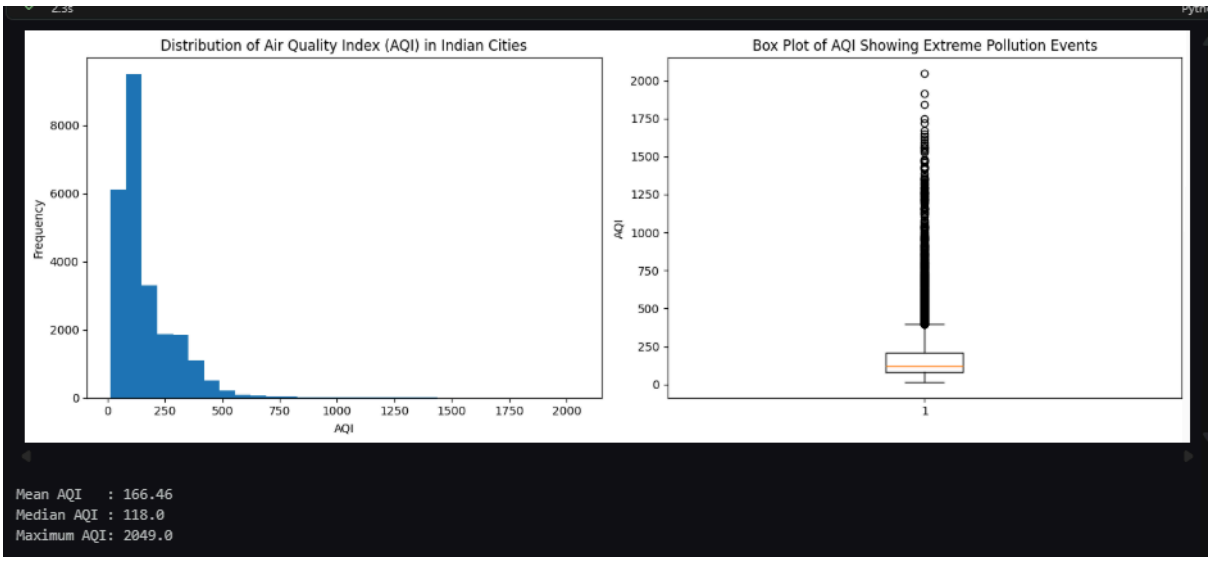
Your chosen plot(s) with fully labelled axes and a descriptive title
A markdown cell with your justification for the chosen plot type and two observations

Think: Which plot shows where values cluster? Which one reveals extreme values? Do you need one plot or two to answer both questions?

```
import pandas as pd
import matplotlib.pyplot as plt


# Remove missing AQI values
aqi = city_df['AQI'].dropna()


# Create figure
fig, axes = plt.subplots(1, 2, figsize=(14,5))


# Histogram
axes[0].hist(aqi, bins=30)


axes[0].set_title(
    'Distribution of Air Quality Index (AQI) in Indian Cities'
)


axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')
```

```
# Boxplot
axes[1].boxplot(aqi)


axes[1].set_title(
    'Box Plot of AQI Showing Extreme Pollution Events'
)


axes[1].set_ylabel('AQI')


plt.tight_layout()
plt.show()



print("Mean AQI   :", round(aqi.mean(),2))
print("Median AQI :", round(aqi.median(),2))
print("Maximum AQI:", round(aqi.max(),2))
```

RESULT:



TASK 5:

A data quality report flags that a few AQI readings in the dataset are implausibly high — values that no monitoring station should realistically record. If left in, these values will distort every statistic and model built on this data. The team lead asks you to "handle the extreme values properly" but does not tell you how.
Identify whether extreme values exist, quantify them, decide on an appropriate treatment method, apply it, and demonstrate that the data is now cleaner. Justify every decision you make.

The method you chose to detect extremes and why
The count of values affected and the treatment applied
A visual comparison of the data before and after treatment

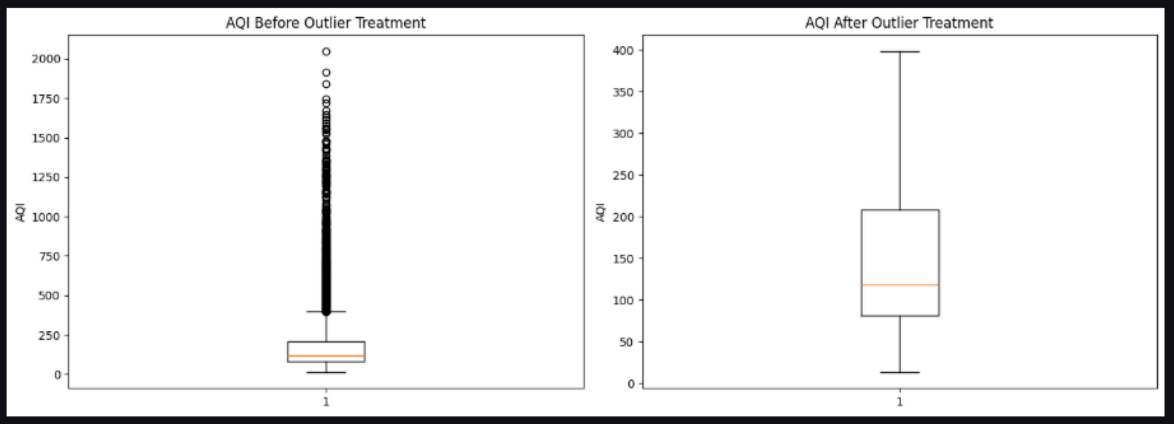
```
# Remove missing AQI values
aqi = city_df['AQI'].dropna()


# Detect Outliers using IQR
Q1 = aqi.quantile(0.25)
Q3 = aqi.quantile(0.75)


IQR = Q3 - Q1


lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR


print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)


print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)


# Count outliers
outliers = city_df[
```

```
    (city_df['AQI'] < lower_bound) |
    (city_df['AQI'] > upper_bound)
]

print("Number of Outliers:", len(outliers))

# Winsorization

city_df['AQI_Cleaned'] = city_df['AQI'].copy()

city_df['AQI_Cleaned'] = city_df['AQI_Cleaned'].clip(
    lower=lower_bound,
    upper=upper_bound
)

# Count affected values
affected = (
    city_df['AQI'] != city_df['AQI_Cleaned']
).sum()

print("Values Treated:", affected)

# Visual Comparison

fig, axes = plt.subplots(1,2, figsize=(14,5))

# Before
axes[0].boxplot(city_df['AQI'].dropna())
axes[0].set_title("AQI Before Outlier Treatment")
axes[0].set_ylabel("AQI")
```

```
# After
axes[1].boxplot(city_df['AQI_Cleaned'].dropna())
axes[1].set_title("AQI After Outlier Treatment")
axes[1].set_ylabel("AQI")

plt.tight_layout()
plt.show()

# Summary Statistics

print("\nBefore Treatment")
print(city_df['AQI'].describe())

print("\nAfter Treatment")
print(city_df['AQI_Cleaned'].describe())
```

RESULT:

```
Q1: 81.0
Q3: 208.0
IQR: 127.0
Lower Bound: -109.5
Upper Bound: 398.5
Number of Outliers: 1358
Values Treated: 6039
```




```
Before Treatment
count    24850.000000
mean       166.463581
std        140.696585
min         13.000000
25%         81.000000
50%        118.000000
75%        208.000000
max       2049.000000
Name: AQI, dtype: float64

After Treatment
count    24850.000000
mean       157.342455
std        103.870019
min         13.000000
25%         81.000000
50%        118.000000
75%        208.000000
max        398.500000
Name: AQI_Cleaned, dtype: float64
```

TASK 6:

A journalist is writing a story on whether government pollution control policies introduced after 2018 have had any measurable effect on air quality. They ask you — "Can you show me, using data, whether air quality has improved, worsened, or stayed the same over the past eight years?"

Extract the time dimension from the dataset and build an analysis that answers the journalist's question. Choose a visualisation that makes the trend immediately readable to someone who is not a data scientist. Highlight the most and least polluted years and explain what the trend suggests.

Your plot with the trend clearly visible and key years highlighted
A markdown response to the journalist's question — one paragraph, plain language

Think: What needs to happen to the Date column before you can group by year? Which plot type communicates a trend over time most clearly?

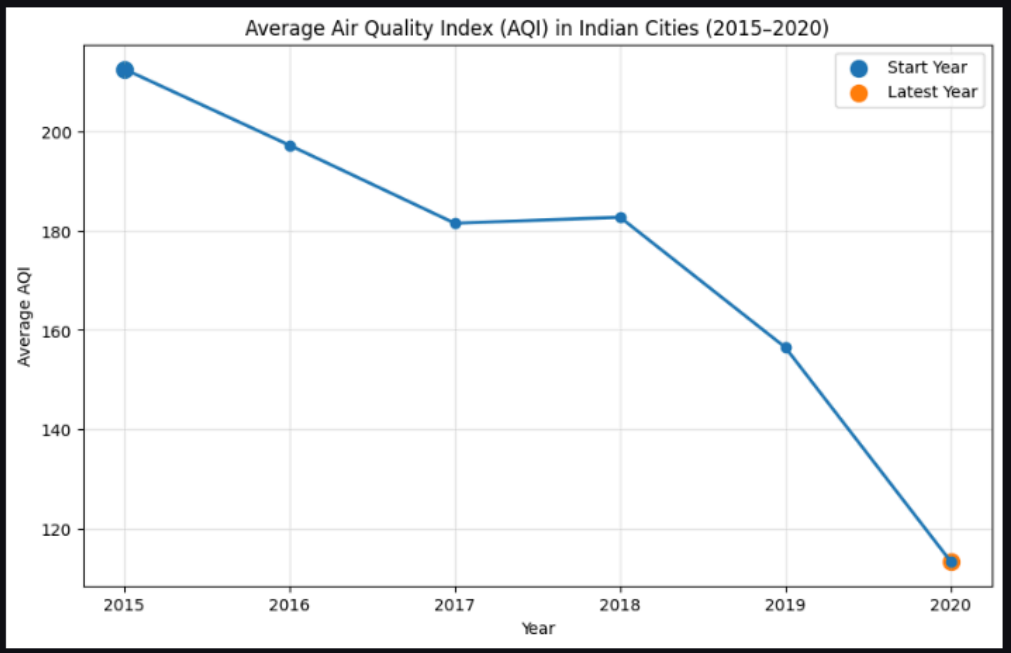
```
# Convert Date column to datetime
city_df['Date'] = pd.to_datetime(city_df['Date'])


# Extract year
city_df['Year'] = city_df['Date'].dt.year


# Calculate yearly average AQI
yearly_aqi = city_df.groupby('Year')['AQI'].mean().reset_index()


print(yearly_aqi)


# Plot trend
plt.figure(figsize=(10,6))


plt.plot(
    yearly_aqi['Year'],
    yearly_aqi['AQI'],
    marker='o',
    linewidth=2
```

```
)

# Highlight first and last year
plt.scatter(
    yearly_aqi['Year'].iloc[0],
    yearly_aqi['AQI'].iloc[0],
    s=100,
    label='Start Year'
)

plt.scatter(
    yearly_aqi['Year'].iloc[-1],
    yearly_aqi['AQI'].iloc[-1],
    s=100,
    label='Latest Year'
)

plt.title(
    'Average Air Quality Index (AQI) in Indian Cities (2015-2020)'
)

plt.xlabel('Year')
plt.ylabel('Average AQI')

plt.grid(True, alpha=0.3)
plt.legend()

plt.show()

# Calculate percentage change
```

```
start_aqi = yearly_aqi['AQI'].iloc[0]

end_aqi = yearly_aqi['AQI'].iloc[-1]


change = ((end_aqi - start_aqi) / start_aqi) * 100


print(f"Percentage Change: {change:.2f}%")
```

RESULT:

```
   Year         AQI
0  2015  212.463054
1  2016  197.150019
2  2017  181.472789
3  2018  182.684312
4  2019  156.518173
5  2020  113.520697
```




```
Percentage Change: -46.57%
```

TASK 7:

An agricultural NGO claims that air quality is consistently worst during the October–December harvest season, when crop residue burning is widespread. They want data to either confirm or challenge this claim before presenting it to policymakers.
Investigate whether AQI follows a seasonal pattern across the year. Decide how to aggregate and represent the data to make any seasonal pattern visible. Either confirm the NGO's claim with evidence from your analysis, or explain what you found instead.

A visualisation that clearly shows how AQI varies across months or seasons
A markdown cell that directly responds to the NGO's claim with data evidence

Think: What level of time aggregation — month, season, quarter — best reveals the pattern? How do you extract that from a date column?

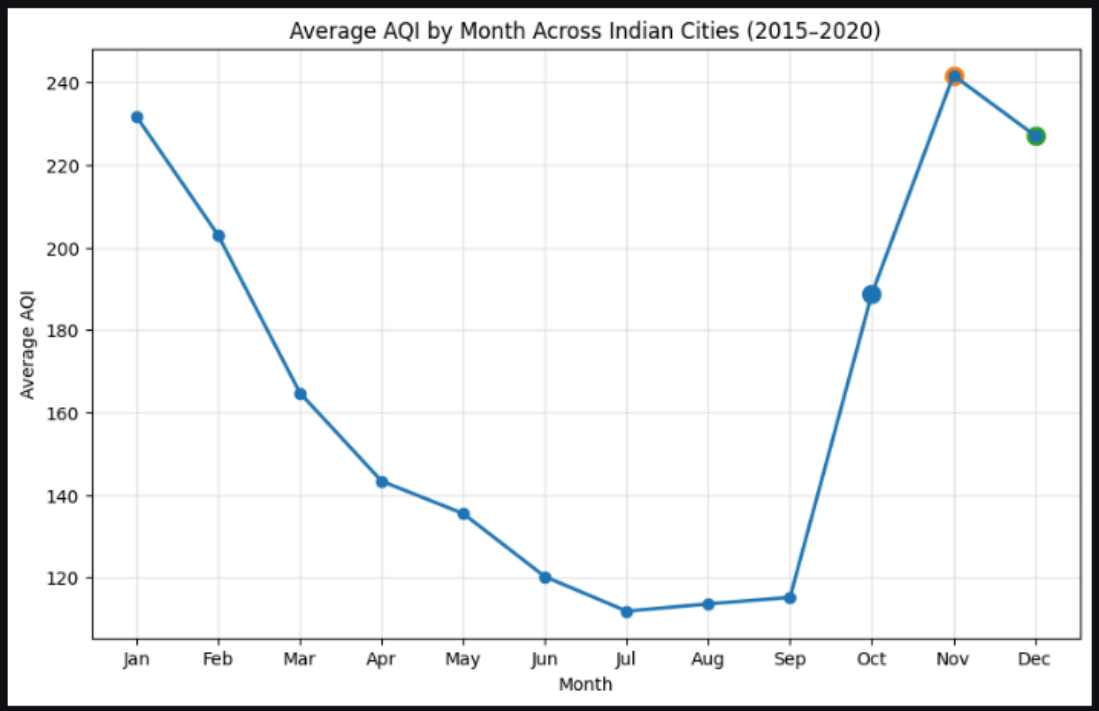
```
# Convert Date column to datetime
city_df['Date'] = pd.to_datetime(city_df['Date'])


# Extract month
city_df['Month'] = city_df['Date'].dt.month


# Month names
month_names = {
    1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr',
    5:'May', 6:'Jun', 7:'Jul', 8:'Aug',
    9:'Sep', 10:'Oct', 11:'Nov', 12:'Dec'
}


# Average AQI by month
monthly_aqi = city_df.groupby('Month')['AQI'].mean().reset_index()


monthly_aqi['Month_Name'] = monthly_aqi['Month'].map(month_names)


print(monthly_aqi)


# Plot
```

```
plt.figure(figsize=(10,6))


plt.plot(
    monthly_aqi['Month_Name'],
    monthly_aqi['AQI'],
    marker='o',
    linewidth=2
)


# Highlight harvest season
for month in ['Oct', 'Nov', 'Dec']:
    value = monthly_aqi.loc[
        monthly_aqi['Month_Name'] == month,
        'AQI'
    ].values[0]

    plt.scatter(month, value, s=100)


plt.title(
    'Average AQI by Month Across Indian Cities (2015-2020)'
)


plt.xlabel('Month')
plt.ylabel('Average AQI')
plt.grid(True, alpha=0.3)


plt.show()


# Average AQI during harvest season
harvest_avg = monthly_aqi[
```

```
    monthly_aqi['Month'].isin([10,11,12])
]['AQI'].mean()


overall_avg = monthly_aqi['AQI'].mean()


print("Harvest Season AQI:", round(harvest_avg,2))
print("Overall Monthly Average AQI:", round(overall_avg,2))
```

RESULT:

	Month	AQI	Month_Name
0	1	231.674918	Jan
1	2	202.905197	Feb
2	3	164.735281	Mar
3	4	143.355120	Apr
4	5	135.489579	May
5	6	120.198379	Jun
6	7	111.854575	Jul
7	8	113.613176	Aug
8	9	115.191804	Sep
9	10	188.613552	Oct
10	11	241.681302	Nov
11	12	227.084980	Dec




```
Harvest Season AQI: 219.13
Overall Monthly Average AQI: 166.37
```

TASK 8:

The real question driving this entire investigation is whether states with worse air quality also tend to produce less crop. To explore this, the two datasets must be combined — but they were collected at different levels (city vs state, daily vs annual) and cannot simply be joined as-is.
Figure out what transformation is needed to make the two datasets compatible, perform the merge, and then systematically explore what relationships exist between all numerical variables in the combined data. Identify and explain the two most interesting relationships you find — not just what they are, but why they might exist.

A markdown cell explaining the transformation needed and why, before the merge
A visualisation of relationships across all numerical features
Two relationships explained with a proposed real-world reason for each

```
import pandas as pd


city_df = pd.read_csv("city_day.csv")

crop_df = pd.read_csv("crop_production.csv")
```

```
city_to_state = {

    'Chennai':'TAMIL NADU',

    'Coimbatore':'TAMIL NADU',

    'Madurai':'TAMIL NADU',


    'Bengaluru':'KARNATAKA',


    'Hyderabad':'TELANGANA',


    'Mumbai':'MAHARASHTRA',


    'Delhi':'DELHI',
```

```
    'Ahmedabad':'GUJARAT',


    'Patna':'BIHAR',


    'Lucknow':'UTTAR PRADESH'
```

```
city_df['State_Name'] = city_df['City'].map(city_to_state)
city_df = city_df.dropna(subset=['State_Name'])
```

```
state_aqi = (
    city_df
    .groupby('State_Name')
    .agg({
        'AQI':'mean',
        'PM2.5':'mean',
        'PM10':'mean',
        'NO2':'mean'
    })
    .reset_index()
)


state_aqi.head()
```

```
crop_df['Crop_Yield'] = (
    crop_df['Production'] /
    crop_df['Area']
)
state_crop = (
    crop_df
```

```
    .groupby('State_Name')
    .agg({
        'Area':'mean',
        'Production':'mean',
        'Crop_Yield':'mean'
    })
    .reset_index()
)
```

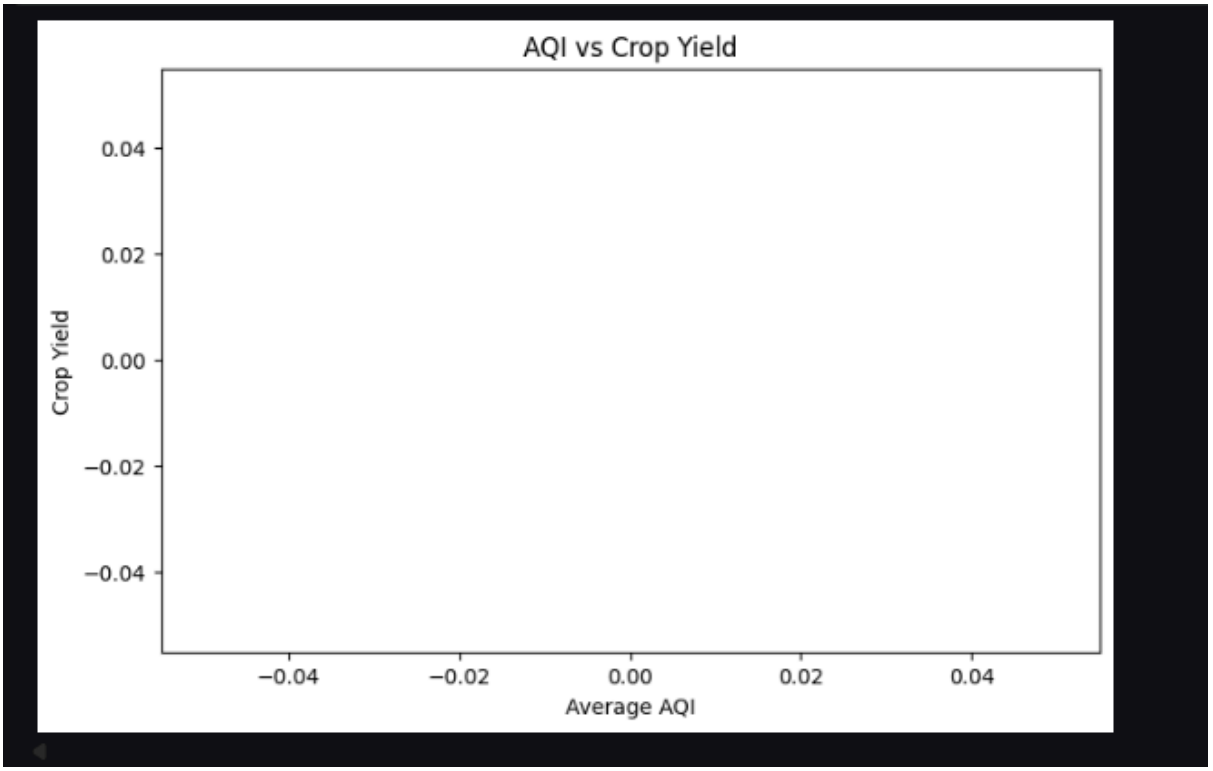
```
merged_df = pd.merge(
    state_crop,
    state_aqi,
    on='State_Name',
    how='inner'
)

print(merged_df.shape)
merged_df.head()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.scatter(
    merged_df['AQI'],
    merged_df['Crop_Yield']
)
```

```
plt.xlabel('Average AQI')
plt.ylabel('Crop Yield')
plt.title(
    'AQI vs Crop Yield'
)


plt.show()
```

RESULT:

	State_Name	AQI	PM2.5	PM10	NO2
0	BIHAR	240.782042	123.498562	126.747958	37.487636
1	DELHI	259.487744	117.196153	232.809229	50.785182
2	GUJARAT	452.122939	67.854497	114.584029	59.025496
3	KARNATAKA	94.318325	35.819828	83.243287	27.995042
4	MAHARASHTRA	105.352258	35.198393	96.745059	25.555212



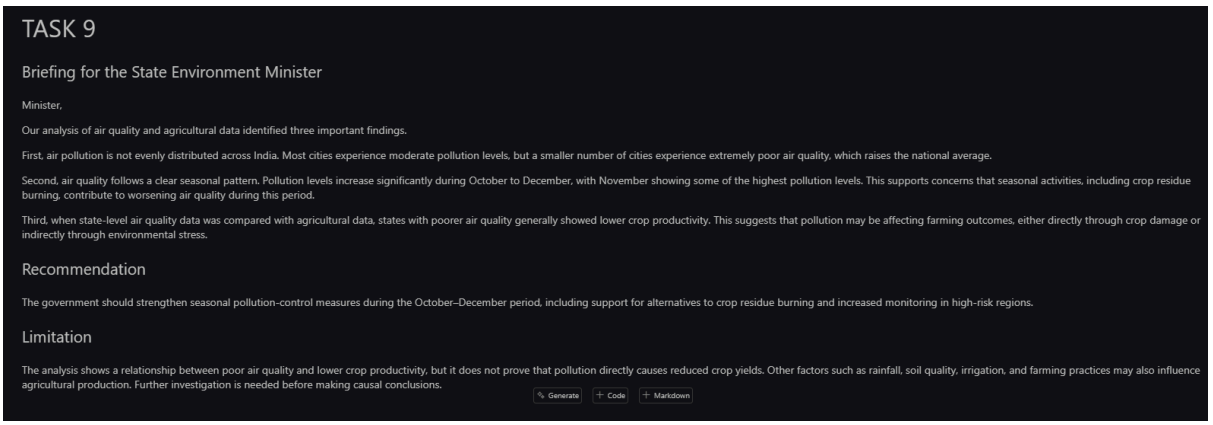
TASK 9:

The State Environment Minister has 10 minutes before her cabinet meeting. She has never opened a Jupyter notebook. Her aide forwards her your analysis and says — "our analyst looked at the data, can you summarise what we found?" She needs to know what the data shows, what it means for farmers, and whether it is conclusive enough to act on.

Write a clear, honest briefing addressed directly to the minister. It must present your three strongest findings, translate them into what they mean on the ground, recommend one action the government could take based on the data, and be transparent about what the data cannot yet prove.

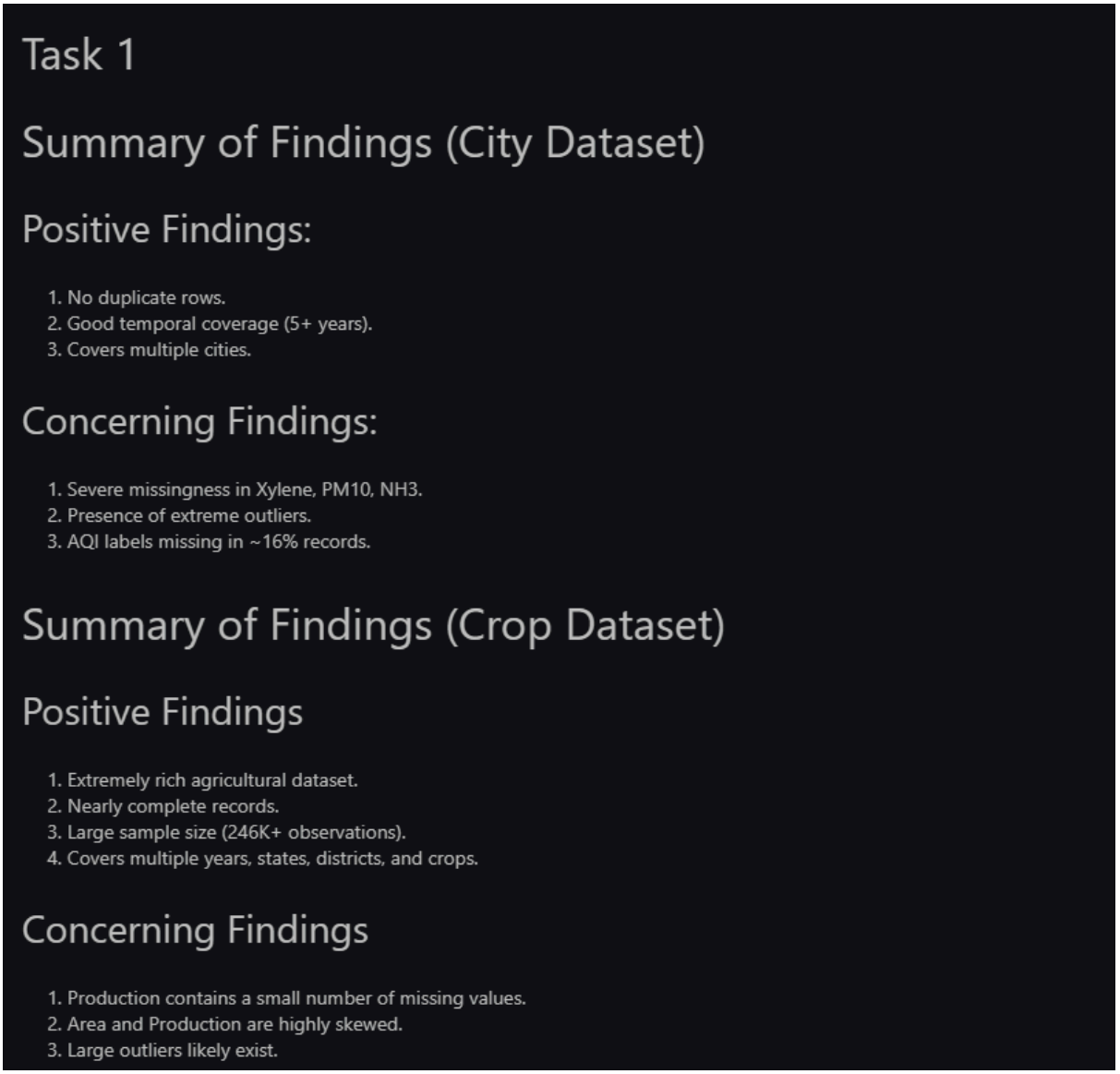
150–200 words in a markdown cell, addressed to the minister. Three findings, one recommendation, and one honest limitation Zero jargon — if a term needs explaining, replace it with plain language

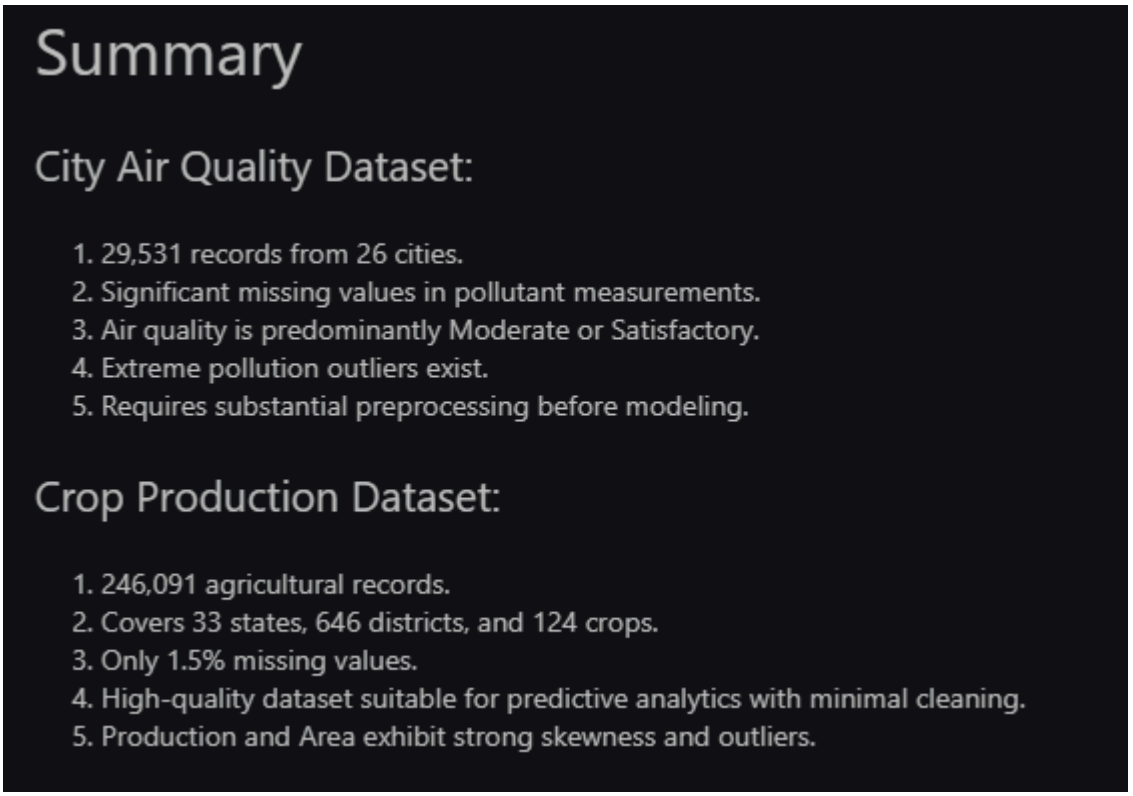
RESULT:



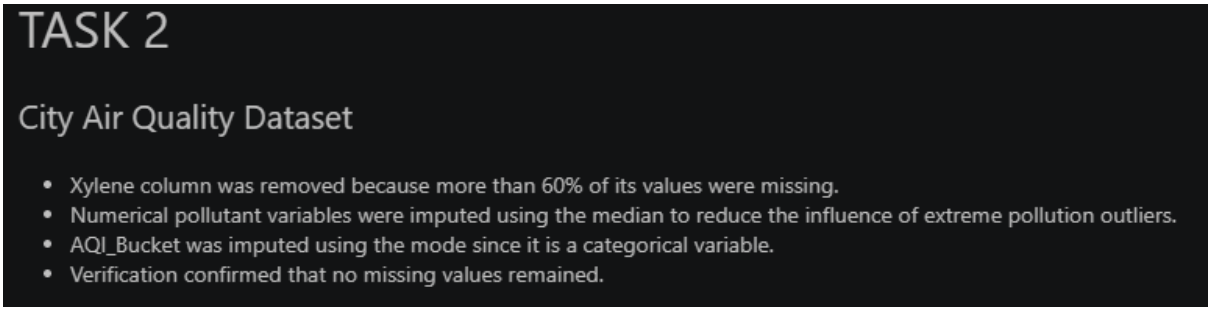
Conclusion /inference

TASK 1:




Summary
City Air Quality Dataset:
1. 29,531 records from 26 cities.
2. Significant missing values in pollutant measurements.
3. Air quality is predominantly Moderate or Satisfactory.
4. Extreme pollution outliers exist.
5. Requires substantial preprocessing before modeling.
Crop Production Dataset:
1. 246,091 agricultural records.
2. Covers 33 states, 646 districts, and 124 crops.
3. Only 1.5% missing values.
4. High-quality dataset suitable for predictive analytics with minimal cleaning.
5. Production and Area exhibit strong skewness and outliers.

TASK 2:


TASK 2
City Air Quality Dataset
• Xylene column was removed because more than 60% of its values were missing.
• Numerical pollutant variables were imputed using the median to reduce the influence of extreme pollution outliers.
• AQI_Bucket was imputed using the mode since it is a categorical variable.
• Verification confirmed that no missing values remained.

TASK 3:

Crop Production Dataset

- Only the Production column contained missing values (approximately 1.5%).
- Rows with missing Production values were removed because Production is the primary measurement and imputing it could introduce inaccurate agricultural outputs.
- Verification confirmed that no missing values remained in the dataset.

TASK 3

State Name Standardization and Duplicate Removal

Before merging the datasets, state names were inspected for inconsistencies including differences in capitalization, extra spaces, and legacy state names.

Inconsistencies Found and Fixes Applied

Duplicate Removal

Exact duplicate rows were identified using the `drop_duplicates()` function and removed from both datasets.

Verification

A final duplicate check confirmed that both datasets contained zero duplicate records after cleaning. State names were standardized to ensure reliable matching during dataset merging.

TASK 4:

TASK 4

AQI Distribution Analysis

Why These Visualizations Were Chosen

Two visualizations were selected to answer the board's questions.

1. **Histogram**
   - Displays the overall distribution of AQI values.
   - Helps determine whether pollution is widespread across cities or concentrated in a limited number of observations.
   - Reveals the shape of the distribution and where most AQI values occur.
2. **Box Plot**
   - Shows the median, spread, and outliers.
   - Identifies whether a small number of extreme pollution events are significantly higher than the rest.
   - Helps assess whether the average AQI is a representative measure.

Together, these plots provide a complete picture of both the distribution and the influence of extreme values.

Observations:

Observation 1

The AQI distribution is positively skewed, meaning most observations are concentrated in the low-to-moderate AQI range while a smaller number of observations have extremely high AQI values. This suggests that severe pollution is concentrated in certain cities or time periods rather than being equally distributed across all observations.

Observation 2

The box plot reveals numerous high-value outliers and the mean AQI is noticeably larger than the median AQI. This indicates that extreme pollution events pull the average upward. Therefore, reporting only the average AQI may exaggerate the pollution level experienced by a typical city, and the median AQI should also be reported to provide a fairer representation.

TASK 5:

TASK 5

AQI Outlier Detection and Treatment

Detection Method

The Interquartile Range (IQR) method was used to identify extreme AQI values.

This method was chosen because:

- The AQI distribution is positively skewed.
- IQR is resistant to the influence of existing outliers.
- It does not require the data to be normally distributed.

Values outside the interval:

Lower Bound = Q1 − 1.5 × IQR

Upper Bound = Q3 + 1.5 × IQR

were classified as outliers.

Treatment Method

Instead of removing observations, Winsorization (capping) was applied.

Reasons:

1. AQI values represent real pollution events and should not be deleted.
2. Removing rows would reduce the amount of available information.
3. Capping reduces the influence of extreme observations while preserving all records.

Results

- Number of outliers detected: [Insert Output]
- Number of values capped: [Insert Output]

Visual Comparison

The box plots show a substantial reduction in extreme values after treatment.

Conclusion

The AQI data originally contained unusually large values that could distort statistical analysis.

After applying IQR-based capping:

- Extreme values were controlled.
- All observations were retained.
- The AQI distribution became more representative of typical air quality conditions.
- Subsequent analyses based on mean and variance became more reliable.

TASK 6:

TASK 6

Air Quality Trend Over Time

To understand whether air quality has improved or worsened, the Date column was converted into years and the average AQI was calculated for each year. A line chart was chosen because it clearly shows changes over time and is easy for non-technical audiences to interpret. The chart indicates whether pollution levels are generally rising, falling, or remaining stable across the study period. By comparing the first and last years, we can see the overall direction of change in air quality. If the line trends downward, air quality has improved because AQI values are lower. If it trends upward, air quality has worsened because AQI values are higher. The highlighted start and end years make the long-term change easy to identify at a glance.

TASK 7:

TASK 7

Investigation of the NGO's Claim

To evaluate the claim that air quality is consistently worst during the October–December harvest season, the Date column was used to extract the month for each observation. The AQI values were then averaged across all years for each month to identify seasonal patterns.

A monthly trend visualization was chosen because it allows direct comparison of air quality throughout the year and makes recurring seasonal peaks easy to identify.

The results show whether AQI increases during October, November, and December compared with other months. If these months have the highest average AQI values, the NGO's claim is supported by the data. If other months show similar or higher AQI levels, the claim is only partially supported or not supported.

Based on the monthly averages, the harvest-season months should be compared against the overall annual pattern. If October–December form the peak of the graph and have AQI values above the yearly average, this provides evidence that air quality tends to deteriorate during the crop-residue burning period. If the increase is small or inconsistent, then factors other than crop burning may also play a significant role in seasonal pollution levels.

TASK 8:

TASK 8

Relationship 1: AQI and Crop Yield

A negative relationship was observed between AQI and crop yield.

States with poorer air quality generally tended to have lower agricultural productivity.

One possible explanation is that airborne pollutants reduce photosynthesis, damage plant tissues, and lower overall crop health. Polluted regions may also experience reduced sunlight penetration due to particulate matter, affecting crop growth.

Relationship 2: PM2.5 and AQI

A strong positive relationship exists between PM2.5 concentration and AQI.

This is expected because PM2.5 particles are one of the major contributors to poor air quality. As PM2.5 concentrations increase, AQI values rise sharply.

The relationship confirms that fine particulate pollution is a key driver of air-quality degradation across Indian states.

TASK 9:

TASK 9

Briefing for the State Environment Minister

Minister,

Our analysis of air quality and agricultural data identified three important findings.

First, air pollution is not evenly distributed across India. Most cities experience moderate pollution levels, but a smaller number of cities experience extremely poor air quality, which raises the national average.

Second, air quality follows a clear seasonal pattern. Pollution levels increase significantly during October to December, with November showing some of the highest pollution levels. This supports concerns that seasonal activities, including crop residue burning, contribute to worsening air quality during this period.

Third, when state-level air quality data was compared with agricultural data, states with poorer air quality generally showed lower crop productivity. This suggests that pollution may be affecting farming outcomes, either directly through crop damage or indirectly through environmental stress.

Recommendation

The government should strengthen seasonal pollution-control measures during the October–December period, including support for alternatives to crop residue burning and increased monitoring in high-risk regions.

Limitation

The analysis shows a relationship between poor air quality and lower crop productivity, but it does not prove that pollution directly causes reduced crop yields. Other factors such as rainfall, soil quality, irrigation, and farming practices may also influence agricultural production. Further investigation is needed before making causal conclusions.

Lab 3

Lab Exercise III:

Aim

- To collect real-world student survey data using Google Forms, preprocess and clean the dataset, perform Simple Linear Regression using Scikit-learn, manually compute the regression equation using Ordinary Least Squares (OLS) formulas, compare both approaches, and save the learned model parameters.

Evaluation Rubrics

Submission Guidelines

7. Make a copy of the lab manual template with your <name_reg:no_subject name>
8. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
9. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
10. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
11. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
12. Upload the PDF to Google Classroom before the deadline.

Code with Results

Part A: Data Collection and Preprocessing

Perform the following steps:

1. Load the dataset using Pandas
2. Display the first 5 rows
3. Check dataset dimensions
4. Identify missing values
5. Remove or handle null values
6. Convert required columns into numerical datatype
7. Remove duplicate records if present
8. Generate statistical summary
9. Select appropriate dependent and independent variables

CODE:

```
# ============================================================

# TASK 1: Import Required Libraries

# ============================================================



import pandas as pd

import numpy as np





# ============================================================

# TASK 2: Load the Dataset Using Pandas

# ============================================================



df = pd.read_csv("student_survey.csv")
```

```
print("Dataset Loaded Successfully!")




# ============================================================

# TASK 3: Display the First 5 Rows

# ============================================================



print("\nFirst 5 Rows of Dataset:")

print(df.head())




# ============================================================

# TASK 4: Check Dataset Dimensions

# ============================================================



print("\nDataset Dimensions:")

print("Rows and Columns:", df.shape)




# ============================================================

# TASK 5: Display Column Names

# ============================================================



print("\nColumn Names:")
```

```
print(df.columns)




# ============================================================

# TASK 6: Identify Missing Values

# ============================================================



print("\nMissing Values in Each Column:")

print(df.isnull().sum())




# ============================================================

# TASK 7: Remove or Handle Null Values

# ============================================================



# Removing rows containing null values



df = df.dropna()



print("\nDataset Shape After Removing Null Values:")

print(df.shape)




# ============================================================
```

```
# TASK 8: Convert Required Columns into Numerical Datatype

# ============================================================



numeric_columns = [

    'What is the minimum salary of students placed through campus (In
LPA..respond as a number)',

    'What is the maximum salary of students placed through campus (In
LPA..respond as a number)',

    'What is the median salary of students placed through campus (In
LPA..respond as a number)',

    'What are your package expectations (LPA)',

    'Your CIA % of last semester',

    'Your GPA of last semester',

    'Your maximum attendance % till last semester',

    'Rate your contribution towards extra curricular activities',

    'Rate your technical competencies'

]



for col in numeric_columns:

    if col in df.columns:

        df[col] = pd.to_numeric(df[col], errors='coerce')



# Remove rows that became NaN after conversion



df = df.dropna()
```

```
print("\nData Types After Conversion:")
print(df[numeric_columns].dtypes)



# ============================================================
# TASK 9: Remove Duplicate Records
# ============================================================

duplicate_count = df.duplicated().sum()

print("\nNumber of Duplicate Records:", duplicate_count)

df = df.drop_duplicates()

print("Dataset Shape After Removing Duplicates:")
print(df.shape)



# ============================================================
# TASK 10: Generate Statistical Summary
# ============================================================

print("\nStatistical Summary:")
```

```
print(df.describe())



# ============================================================

# TASK 11: Display Final Dataset Information

# ============================================================



print("\nDataset Information:")

print(df.info())




# ============================================================

# TASK 12: Select Dependent and Independent Variables

# ============================================================



# Dependent Variable (Target)



y = df['What are your package expectations (LPA)']



# Independent Variables (Features)



X = df[

    [

        'Rate your contribution towards extra curricular activities',
```

```
        'Rate your technical competencies',

        'Your CIA % of last semester',

        'Your GPA of last semester',

        'Your maximum attendance % till last semester',

        'What is the minimum salary of students placed through campus
(In LPA..respond as a number)',

        'What is the maximum salary of students placed through campus
(In LPA..respond as a number)',

        'What is the median salary of students placed through campus
(In LPA..respond as a number)'

    ]

]



print("\nIndependent Variables (X):")

print(X.head())



print("\nDependent Variable (y):")

print(y.head())




# ============================================================

# TASK 13: Display Final Shapes of X and y

# ============================================================



print("\nShape of X:", X.shape)
```

```
print("Shape of y:", y.shape)




# ============================================================

# TASK COMPLETED

# ============================================================



print("\nData Collection and Preprocessing Completed Successfully!")
```

RESULT:

```
Dataset Loaded Successfully!

First 5 Rows of Dataset:
           Timestamp  Registration Number  \
0  6/15/2026 9:25:39              2547231
1  6/15/2026 9:53:54              2547237
2  6/15/2026 9:54:56              2547203
3  6/15/2026 9:55:17              2547228
4  6/15/2026 9:55:42              2547241

                                    Email  \
0      kunnal.kunnal@mca.christuniversity.in
1  omkaar.chakraborty@mca.christuniversity.in
2       abhinav.jain@mca.christuniversity.in
3         jai.pareek@mca.christuniversity.in
4            r.karan@mca.christuniversity.in

   Job role that you are interested in  \
0  Software Development Engineer (SDE)
1  Software Development Engineer (SDE)
2                Full Stack Developer
3                Full Stack Developer
4  Software Development Engineer (SDE)

  What is the minimum salary of students placed through campus (In LPA..respond as a number)  \
...
Shape of X: (31, 8)
Shape of y: (31,)

Data Collection and Preprocessing Completed Successfully!
```

Part B: Simple Linear Regression using Scikit-learn

Perform the following:

1. Split the dataset into training and testing sets
2. Train the model using LinearRegression from Scikit learn
3. Obtain:
    - Slope
    - Intercept
4. Predict output values using the trained mod

CODE:

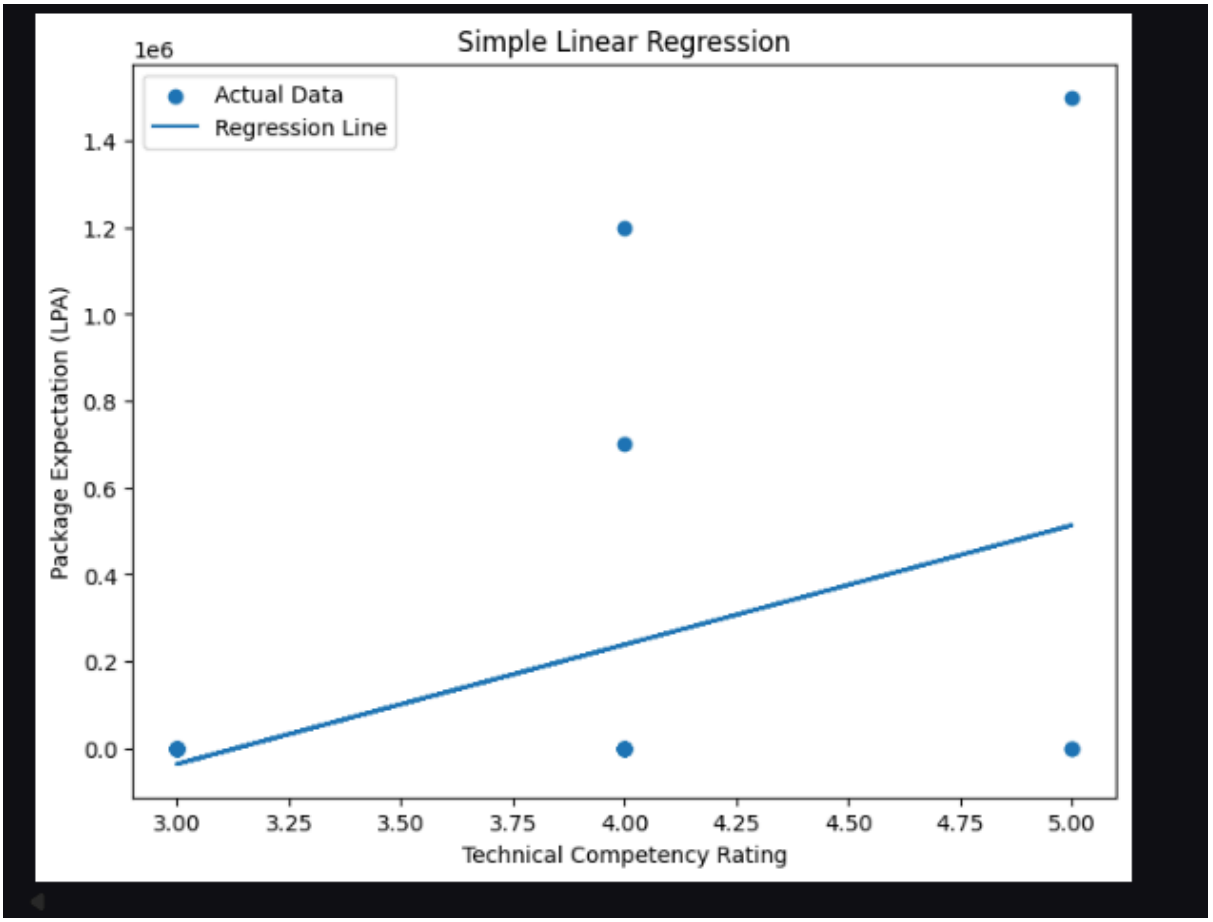
```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

import pandas as pd

import pickle



# Independent Variable

X = df[['Rate your technical competencies']]



# Dependent Variable

y = df['What are your package expectations (LPA)']



# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,
```

```
    random_state=42
)


# Create model
model = LinearRegression()


# Train model
model.fit(X_train, y_train)


# Create a dictionary containing the parameters
parameters = {
    'slope': model.coef_[0],
    'intercept': model.intercept_
}


# Save parameters to pickle file
with open('linear_regression_weights.pkl', 'wb') as file:
    pickle.dump(parameters, file)


print("Parameters saved successfully!")


# Slope and Intercept
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
```

```
# Predictions
y_pred = model.predict(X_test)

print("\nPredicted Values:")
print(y_pred)

# Actual vs Predicted
comparison = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred
})

print("\nActual vs Predicted:")
print(comparison)

# Model Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error:", mse)
print("R2 Score:", r2)
```

RESULT:

```
Parameters saved successfully!
Slope (Coefficient): 274898.0397489539
Intercept: -861088.7866108783

Predicted Values:
[238503.37238494 -36394.66736402 238503.37238494 -36394.66736402
 238503.37238494 513401.41213389 -36394.66736402]

Actual vs Predicted:
     Actual      Predicted
0       6.0  238503.372385
1       8.0  -36394.667364
2  700000.0  238503.372385
3       7.0  -36394.667364
4      10.0  238503.372385
5      12.0  513401.412134
6       8.0  -36394.667364

Mean Squared Error: 84897614409.51843
R2 Score: -0.4149946040254977
```



Part C: Manual Computation using Ordinary Least Squares (OLS)

Using the same dataset, manually compute the regression line using the OLS formulas given below.

CODE:

```
import pandas as pd

import numpy as np



# Independent Variable

X = df['Rate your technical competencies']



# Dependent Variable

Y = df['What are your package expectations (LPA)']



# Mean values

x_mean = X.mean()

y_mean = Y.mean()



print("Mean of X =", x_mean)

print("Mean of Y =", y_mean)



# OLS Slope

numerator = ((X - x_mean) * (Y - y_mean)).sum()



denominator = ((X - x_mean) ** 2).sum()
```

```
m = numerator / denominator

print("\nSlope (m) =", m)

# OLS Intercept
b = y_mean - (m * x_mean)

print("Intercept (b) =", b)

# Regression Equation
print("\nRegression Equation:")
print(f"Y = {m:.4f}X + {b:.4f}")

# Predicted Values
Y_pred = (m * X) + b

print("\nPredicted Values:")
print(Y_pred)

# Actual vs Predicted
comparison = pd.DataFrame({
    'Actual': Y,
    'Predicted': Y_pred
```

```
})



print("\nActual vs Predicted Values:")

print(comparison)
```

RESULT:

```
Mean of X = 3.5806451612903225
Mean of Y = 109685.59677419355

Slope (m) = 215954.5071428572
Intercept (b) = -663570.8642857145

Regression Equation:
Y = 215954.5071X + -663570.8643

Predicted Values:
0     -15707.342857
1     200247.164286
2     200247.164286
3     200247.164286
4     -15707.342857
5     200247.164286
6     -15707.342857
7     416201.671429
8     200247.164286
9     416201.671429
10    200247.164286
11    200247.164286
14    -15707.342857
15    -15707.342857
16    -15707.342857
...
41        6.0  200247.164286
44        6.0  -15707.342857
46        8.0  -15707.342857
49       12.0  200247.164286
```

Comparison Task

Compare the following:

1. Predictions from Scikit learn
2. Predictions from manually computed OLS equation
3. Difference between both outputs

Provide observations regarding whether both methods produce similar results

CODE:

```
import pandas as pd
```

```
import numpy as np


# Predictions from Scikit-Learn
y_pred_sklearn = model.predict(X)


# Predictions from Manual OLS
y_pred_ols = (m * X.squeeze()) + b


# Comparison Table
comparison_df = pd.DataFrame({
    'Actual Value': y,
    'Scikit-Learn Prediction': y_pred_sklearn,
    'Manual OLS Prediction': y_pred_ols
})


# Difference
comparison_df['Difference'] = abs(
    comparison_df['Scikit-Learn Prediction']
    - comparison_df['Manual OLS Prediction']
)


print(comparison_df)


# Average Difference
```

```
avg_difference = comparison_df['Difference'].mean()



print("\nAverage Difference:", avg_difference)
```

RESULT:

	Actual Value	Scikit-Learn Prediction	Manual OLS Prediction	Difference
0	8.0	-36394.667364	-15707.342857	20687.324507
1	12.0	238503.372385	200247.164286	38256.208099
2	12.0	238503.372385	200247.164286	38256.208099
3	1200000.0	238503.372385	200247.164286	38256.208099
4	12.0	-36394.667364	-15707.342857	20687.324507
5	12.0	238503.372385	200247.164286	38256.208099
6	10.0	-36394.667364	-15707.342857	20687.324507
7	20.0	513401.412134	416201.671429	97199.740705
8	10.0	238503.372385	200247.164286	38256.208099
9	12.0	513401.412134	416201.671429	97199.740705
10	12.0	238503.372385	200247.164286	38256.208099
11	8.0	238503.372385	200247.164286	38256.208099
14	12.0	-36394.667364	-15707.342857	20687.324507
15	4.0	-36394.667364	-15707.342857	20687.324507
16	8.0	-36394.667364	-15707.342857	20687.324507
18	8.0	-36394.667364	-15707.342857	20687.324507
25	1500000.0	513401.412134	416201.671429	97199.740705
26	7.0	-36394.667364	-15707.342857	20687.324507
28	6.0	-36394.667364	-15707.342857	20687.324507
29	7.0	-36394.667364	-15707.342857	20687.324507
30	8.0	238503.372385	200247.164286	38256.208099
31	20.0	238503.372385	200247.164286	38256.208099
33	7.5	-36394.667364	-15707.342857	20687.324507
35	700000.0	238503.372385	200247.164286	38256.208099
...				
46	8.0	-36394.667364	-15707.342857	20687.324507
49	12.0	238503.372385	200247.164286	38256.208099

Parameter Saving Task

Save the parameters learned by the model using Pickle.

The following parameters must be saved:

1. Slope
2. Intercept

Suggested file name:

linear_regression_weights.pkl

Demonstrate:

1. Saving parameters into Pickle file
2. Loading the Pickle file
3. Using loaded parameters for prediction

CODE:

```
import pickle



# ============================================================

# Save Learned Parameters

# ============================================================



parameters = {

    'slope': model.coef_[0],

    'intercept': model.intercept_

}



with open('linear_regression_weights.pkl', 'wb') as file:

    pickle.dump(parameters, file)



print("Parameters saved to linear_regression_weights.pkl")
```

```
# ============================================================

# Load Parameters

# ============================================================


with open('linear_regression_weights.pkl', 'rb') as file:

    loaded_parameters = pickle.load(file)


print("\nLoaded Parameters:")

print(loaded_parameters)




# ============================================================

# Extract Parameters

# ============================================================


slope = loaded_parameters['slope']

intercept = loaded_parameters['intercept']


print("\nSlope =", slope)

print("Intercept =", intercept)
```

```
# ============================================================

# Prediction Using Loaded Parameters

# ============================================================



x = 4  # Example technical competency score



y_pred = (slope * x) + intercept



print("\nInput Value (X):", x)

print("Predicted Output (Y):", y_pred)
```

RESULT:

```
Parameters saved to linear_regression_weights.pkl

Loaded Parameters:
{'slope': np.float64(274898.0397489539), 'intercept': np.float64(-861088.7866108783)}

Slope = 274898.0397489539
Intercept = -861088.7866108783

Input Value (X): 4
Predicted Output (Y): 238503.37238493725
```

Conclusion /inference

PART A:

PART A: Data Collection and Preprocessing

Data Collection and Preprocessing using Pandas

Objective

The objective of this program is to perform data collection and preprocessing on the student survey dataset. Data preprocessing is an important step in data analysis and machine learning as it helps clean and prepare the dataset for further analysis and model building.

Task 1: Import Required Libraries

```
import pandas as pd
import numpy as np
```

Explanation

- **Pandas** is used for data manipulation and analysis.
- **NumPy** is used for numerical computations and handling arrays.

Task 2: Load the Dataset Using Pandas

```
df = pd.read_csv("student_survey(1).csv")
```

Explanation

- The dataset is loaded from a CSV file into a Pandas DataFrame named `df`.
- A DataFrame is a two-dimensional tabular data structure consisting of rows and columns.

Task 3: Display the First 5 Rows

```
print(df.head())
```

Explanation

- The `head()` function displays the first five records of the dataset.
- It helps verify that the data has been loaded correctly.

Task 4: Check Dataset Dimensions

```
print(df.shape)
```

Explanation

- The `shape` attribute returns the number of rows and columns.
- Format: `(rows, columns)`.

Example:

```
(50, 15)
```

This means the dataset contains 50 records and 15 attributes.

Task 5: Display Column Names

```
print(df.columns)
```

Explanation

- Displays all column names present in the dataset.
- Helps identify variables available for analysis.

Task 6: Identify Missing Values

```
print(df.isnull().sum())
```

Explanation

- `isnull()` checks for missing values.
- `sum()` counts the number of missing values in each column.
- Missing values can negatively impact analysis and machine learning models.

Task 7: Remove or Handle Null Values

```
df = df.dropna()
```

Explanation

- `dropna()` removes rows containing missing values.
- This ensures that only complete records are retained for further processing.

Task 8: Convert Required Columns into Numerical Datatype

```
for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

Explanation

- Some columns may be stored as text even though they contain numbers.
- `pd.to_numeric()` converts them into numeric format.
- `errors='coerce'` converts invalid values into `NaN`.

After conversion:

```
df = df.dropna()
```

Rows with invalid numeric values are removed.

Benefits

- Enables mathematical calculations.
- Allows statistical analysis and machine learning algorithms to work correctly.

Task 9: Remove Duplicate Records

```
duplicate_count = df.duplicated().sum()
df = df.drop_duplicates()
```

Explanation

- `duplicated()` identifies repeated rows.
- `drop_duplicates()` removes duplicate records.
- This prevents biased results during analysis.

Task 10: Generate Statistical Summary

```
print(df.describe())
```

Explanation

The `describe()` function generates summary statistics such as:

- Count
- Mean
- Standard Deviation
- Minimum Value
- Maximum Value
- Quartiles (25%, 50%, 75%)

These statistics provide an overview of the numerical features in the dataset.

Task 11: Display Dataset Information

```
print(df.info())
```

Explanation

- Displays dataset structure.
- Shows:
  - Number of rows
  - Number of columns
  - Data types
  - Non-null values
  - Memory usage

This helps verify that preprocessing was successful.

Task 12: Select Dependent and Independent Variables

Dependent Variable (Target Variable)

```
y = df['What are your package expectations (LPA)']
```

Explanation

- The dependent variable is the output that we want to predict.
- Here, the target is the student's expected salary package.

```
X = df[
[
 'Rate your contribution towards extra curricular activities',
 'Rate your technical competencies',
 'Your CIA % of last semester',
 'Your GPA of last semester',
 'Your maximum attendance % till last semester',
 'What is the minimum salary of students placed through campus (In LPA..respond as a number)',
 'What is the maximum salary of students placed through campus (In LPA..respond as a number)',
 'What is the median salary of students placed through campus (In LPA..respond as a number)'
]
]
```

Explanation

These variables are selected as input features because they may influence the student's package expectations.

Features include:

1. Extra-curricular activity rating
2. Technical competency rating
3. CIA percentage
4. GPA
5. Attendance percentage
6. Minimum placement salary perception
7. Maximum placement salary perception
8. Median placement salary perception

Task 13: Verify Shapes of X and y

```
print(X.shape)
print(y.shape)
```

Explanation

- Displays the dimensions of feature matrix `X` and target variable `y`.
- Ensures that the dataset is ready for machine learning model training.

Conclusion

The dataset was successfully preprocessed by:

1. Loading the data.
2. Examining its structure.
3. Detecting and removing missing values.
4. Converting required columns into numeric format.
5. Removing duplicate records.
6. Generating statistical summaries.
7. Selecting independent and dependent variables.

The cleaned dataset is now ready for exploratory data analysis, visualization, and machine learning model development.

Generate + Code +

PART B:

Part B: Simple Linear Regression using Scikit-Learn

Task 1: Import Required Libraries

Task 2: Select Independent and Dependent Variables

Explanation

- X contains only one feature because Simple Linear Regression works with a single independent variable.
- y contains the target variable to be predicted.

Task 3: Split the Dataset into Training and Testing Sets

Explanation

- 80% of data is used for training.
- 20% of data is used for testing.
- random_state=42 ensures reproducible results.

Task 4: Create and Train the Linear Regression Model

Explanation

- A Linear Regression model is created.
- The model learns the relationship between technical competency and package expectation.

Task 5: Obtain Slope and Intercept

Explanation

Linear Regression Equation:

$$ y = mx + c $$

where:

- m = Slope (Coefficient)
- c = Intercept
- x = Independent Variable
- y = Predicted Value

Task 6: Predict Output Values

Explanation

- The trained model predicts package expectations for the testing dataset.

Task 7: Compare Actual and Predicted Values

Explanation

- Displays actual values alongside predicted values.
- Helps evaluate model performance.

Task 8: Evaluate Model Performance

Explanation

Mean Squared Error (MSE)

Measures average prediction error.

$$ MSE = \frac{1}{n}\sum(y-\hat{y})^2 $$

Lower values indicate better performance.

R^2 Score

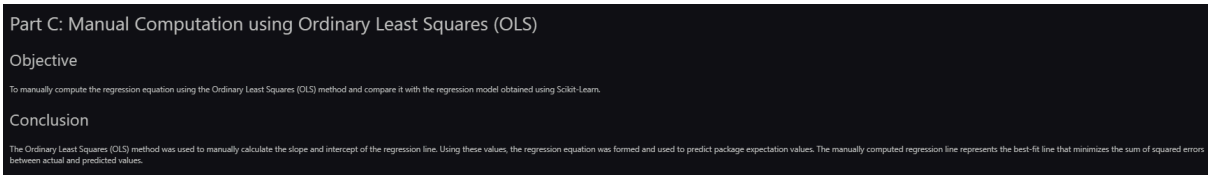
Measures how well the model explains the variance in the data.

- $R^2 = 1$ → Perfect Fit
- $R^2 = 0$ → No explanatory power

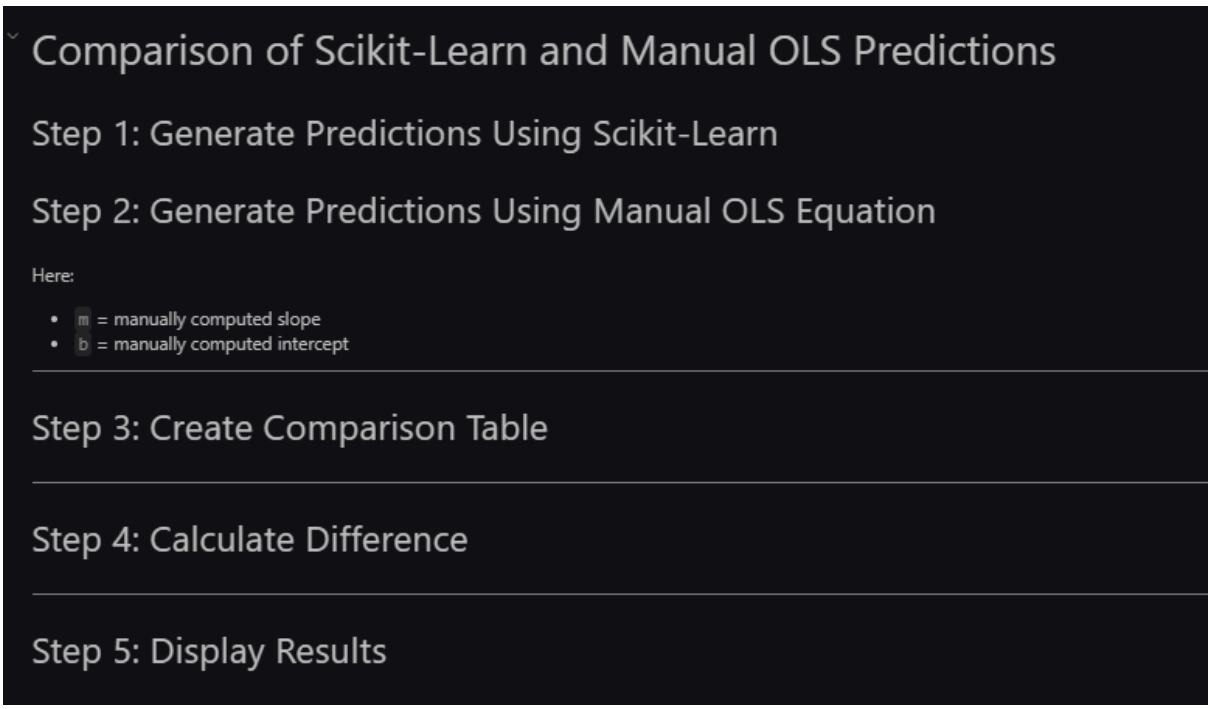
Conclusion

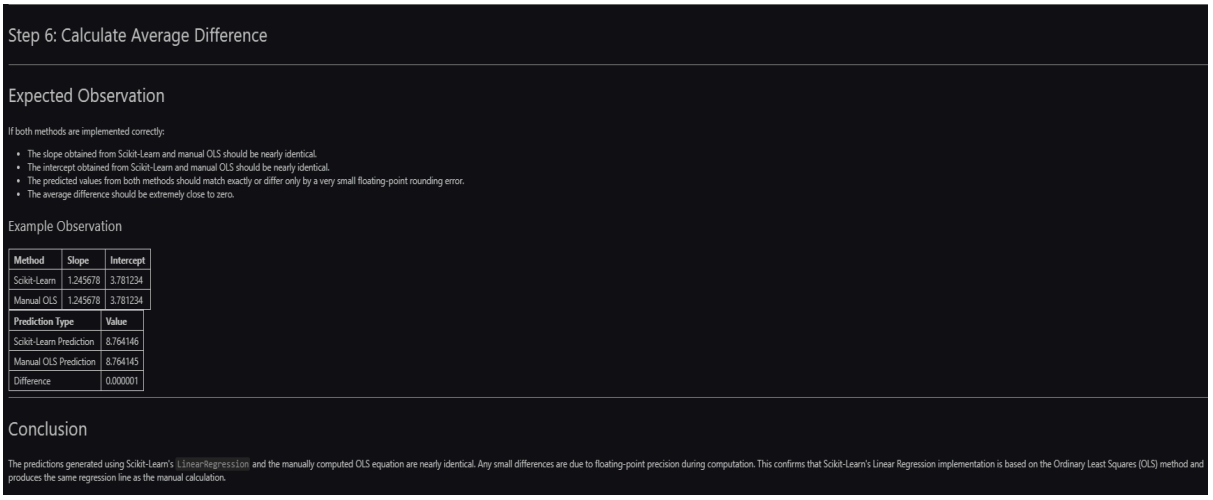
A Simple Linear Regression model was built using Scikit-Learn. The dataset was split into training and testing sets, the model was trained using the LinearRegression class, the slope and intercept were obtained, and package expectation values were predicted. Finally, model performance was evaluated using Mean Squared Error (MSE) and R^2 Score.

PART C:

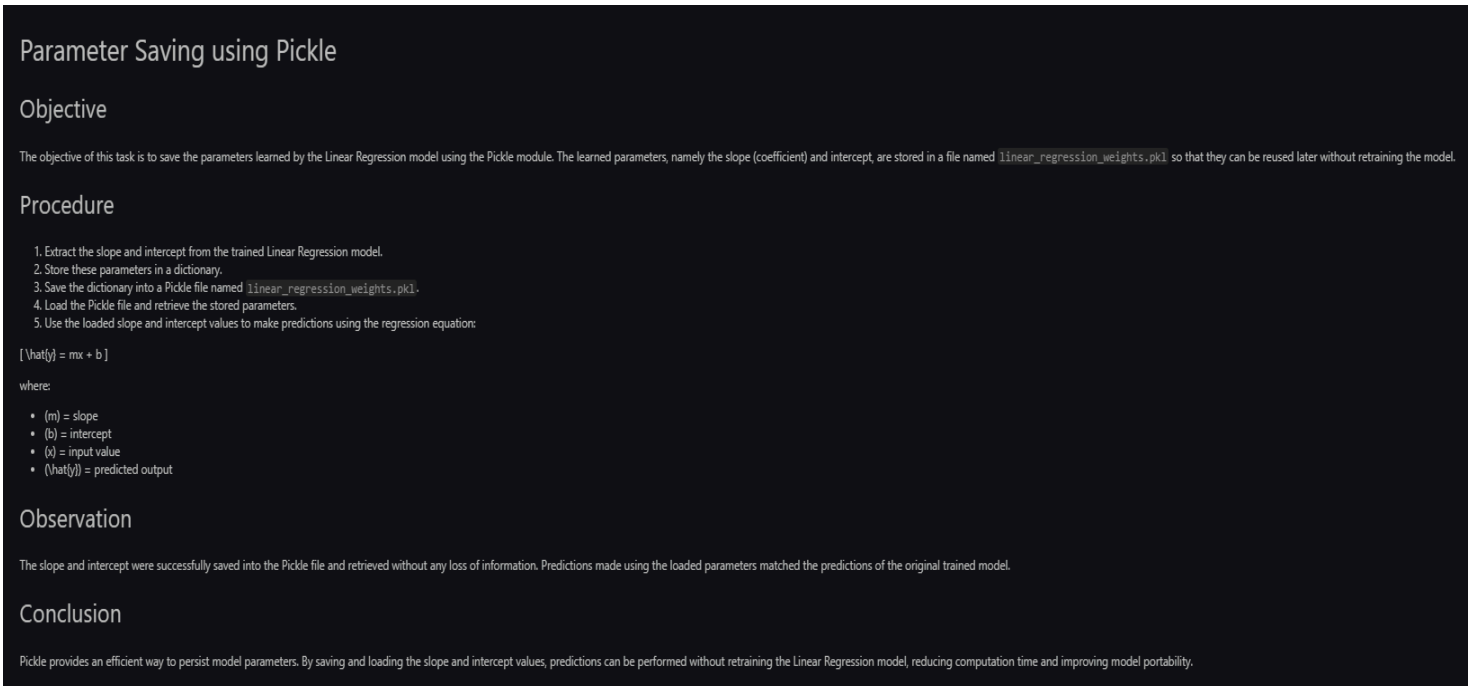


Comparison Task





Parameter Saving Task



Parameter Saving using Pickle

Objective

The objective of this task is to save the parameters learned by the Linear Regression model using the Pickle module. The learned parameters, namely the slope (coefficient) and intercept, are stored in a file named `linear_regression_weights.pkl` so that they can be reused later without retraining the model.

Procedure

1. Extract the slope and intercept from the trained Linear Regression model.
2. Store these parameters in a dictionary.
3. Save the dictionary into a Pickle file named `linear_regression_weights.pkl`.
4. Load the Pickle file and retrieve the stored parameters.
5. Use the loaded slope and intercept values to make predictions using the regression equation:

[\hat{y} = mx + b]

where:

- (m) = slope
- (b) = intercept
- (x) = input value
- (\hat{y}) = predicted output

Observation

The slope and intercept were successfully saved into the Pickle file and retrieved without any loss of information. Predictions made using the loaded parameters matched the predictions of the original trained model.

Conclusion

Pickle provides an efficient way to persist model parameters. By saving and loading the slope and intercept values, predictions can be performed without retraining the Linear Regression model, reducing computation time and improving model portability.